

AI-Assisted Public Health Surveillance Platform: Technical Whitepaper

Research whitepaper for a synthetic AI/ML public health surveillance prototype

Synthetic data only. Not for clinical or operational public health decision-making.

Executive Summary

The AI-Assisted Public Health Surveillance Platform is a full-stack research prototype for syndromic surveillance, anomaly detection, short-horizon forecasting, HL7/EHR-style message processing, retrieval-augmented public health guidance, explainable insight generation, and investigation brief production. It is designed to demonstrate how modern AI/ML methods can support public health analysts who need to distinguish meaningful early signals from data quality artifacts and routine operational noise.

The platform uses a deterministic simulation engine to generate synthetic surveillance data across public health regions, facilities, syndrome categories, age groups, and reporting conditions. The backend transforms those records into engineered surveillance features, applies multiple anomaly detection methods, produces near-term forecasts, indexes a local public health knowledge base, and exposes structured API endpoints. The frontend presents this information in analyst-oriented workflows: a surveillance dashboard, HL7/EHR processing view, ML signal detection table, forecasting page, knowledge retrieval panel, explainable insights page, and investigation brief page.

The project is intentionally framed as decision support rather than automation. It does not confirm outbreaks, does not use real patient data, and does not connect to operational health systems. Its purpose is to provide a transparent architecture for research, demonstration, portfolio evaluation, and future validation.

1. Research Context and Problem Statement

Public health surveillance depends on timely interpretation of heterogeneous, imperfect, and often delayed data. Syndromic systems commonly monitor emergency department visits, chief complaints, diagnosis codes, test positivity, hospitalizations, deaths, facility reporting patterns, and demographic strata. These indicators rarely move at the same time. Test positivity may increase before ED volume. Hospitalizations and deaths may lag. Facility-level reporting failures may create artificial spikes. Small facilities may produce volatile baselines.

The operational challenge is not simply detecting a high count. Analysts must decide whether a signal is:

- a plausible early outbreak,
- a severity signal,
- a facility reporting artifact,
- a coding or data quality issue,
- or an undetermined pattern requiring further review.

This project asks a research-oriented system design question:

How can a public health surveillance application combine transparent ML evidence, data quality context, indicator timing, and retrieval-augmented guidance into an analyst-centered workflow?

The implementation demonstrates one answer. It combines reproducible synthetic data, interpretable statistical baselines, unsupervised anomaly detection, short-term forecasting, local RAG, and structured explanations.

2. System Objectives

The platform is built around six objectives:

- 1 Simulate realistic syndromic surveillance data without using patient records.
- 2 Detect unusual patterns across region, facility, syndrome, and age-group strata.
- 3 Separate probable data quality artifacts from plausible epidemiologic signals.
- 4 Forecast short-term ED visit activity with interpretable uncertainty bounds.
- 5 Retrieve public health guidance relevant to the signal being reviewed.
- 6 Produce explainable summaries and investigation briefs suitable for analyst review.

The system is not intended to optimize a single benchmark metric. Instead, it emphasizes interpretability, workflow realism, reproducibility, and transparent model evidence.

3. Application Architecture

The architecture follows a modular surveillance workflow:

```
Synthetic HL7/EHR-style data
-> HL7 message generation and parsing
-> surveillance metric persistence
-> feature engineering
-> anomaly detection and forecasting
-> RAG knowledge retrieval
-> explainable insight generation
-> investigation brief generation
-> analyst-facing frontend
```

The backend is a FastAPI application. SQLAlchemy models represent facilities, raw synthetic HL7 messages, parsed encounters, surveillance metrics, anomaly results, RAG documents, and generated reports. The frontend is a Next.js application with TypeScript, TailwindCSS, Recharts, and compact UI primitives.

3.1 Frontend

The frontend provides analyst workflows rather than a generic dashboard. Each page maps to a surveillance task:

- Landing page: project orientation and workflow entry points.
- Surveillance Dashboard: descriptive monitoring by region, facility, syndrome, and age group.
- HL7/EHR Processing: raw message review, parser output, and data quality context.
- ML Signal Detection: model-ranked anomaly table with filters and evidence columns.
- Forecasting: 7-day and 14-day ED visit projections with uncertainty bands.
- Knowledge Retrieval: public health RAG panel with supporting snippets.
- Explainable Insights: structured anomaly interpretation.
- Investigation Brief: generated narrative report for synthetic signal review.

3.2 Backend

The backend exposes API endpoints for data generation, dashboard aggregation, HL7 parsing, anomaly detection, forecasting, RAG query, RAG indexing, insight generation, and report generation. The backend is deliberately simple enough to inspect: most core research logic lives in service modules under `backend/app/services`.

3.3 Data Layer

The application supports SQLite for local development and PostgreSQL through Docker Compose. The local path is useful for rapid experimentation; PostgreSQL is more representative of deployment patterns. SQLAlchemy provides a consistent data access layer.

4. Technical Stack

Layer	Technology	Role
Web framework	Next.js, React, TypeScript	Analyst-facing UI and routing
Styling	TailwindCSS	Responsive layout and visual system
Visualization	Recharts	Time series, bar charts, forecast intervals
API framework	FastAPI	REST endpoints and interactive docs
Data models	Pydantic, SQLAlchemy	Validation and persistence
Database	SQLite, PostgreSQL	Local and containerized storage
ML stack	pandas, NumPy, scikit-learn	Feature engineering and models
Vector store	ChromaDB	Local semantic retrieval
Optional AI	OpenAI embeddings and generation	RAG answer synthesis and style control
Deployment	Docker, Docker Compose	Local multi-service deployment

The stack was selected for transparency, speed of iteration, and reproducibility. The ML layer uses established tabular methods rather than opaque deep models so that evidence can be inspected at the row level.

5. Synthetic Surveillance Simulation

The platform uses a deterministic SurveillanceSimulationEngine to generate surveillance records. The simulation is not random filler data. It encodes properties that are common in public health surveillance:

- facility size differences,
- regional variation,
- day-of-week effects,
- syndrome-specific baselines,
- seasonal respiratory patterns,
- warmer-period heat-related illness patterns,

- stable injury baselines,
- small-facility noise,
- reporting delays,
- missing fields,
- duplicate messages,
- facility downtime,
- batch uploads,
- and inconsistent labels.

5.1 Dataset Scope

The default synthetic dataset includes:

- 180 days,
- 5 public health regions,
- 20 facilities,
- 8 syndrome groups,
- 5 age groups,
- ED visits,
- test positivity,
- hospitalizations,
- deaths,
- reporting delay,
- data quality score,
- raw HL7-style messages,
- parsed encounters,
- and surveillance metric rows.

The default surveillance metric grid is:

$180 \text{ days} \times 20 \text{ facilities} \times 8 \text{ syndromes} \times 5 \text{ age groups} = 144,000 \text{ rows}$

5.2 Regions

The current synthetic geography uses meaningful public health region labels:

- North Metro,
- South Valley,
- Central Plains,
- East River,
- West Coastal.

These names are intentionally generic and synthetic. They help analysts reason about regional comparisons without implying real locations.

5.3 Injected Scenarios

The simulation includes injected scenarios so the model layer can be evaluated against known signal types.

Respiratory outbreak in Central Plains. Test positivity increases first, ED visits rise several days later, hospitalizations lag ED visits, and deaths lag hospitalizations. Children and older adults are affected more strongly.

Gradual gastrointestinal increase in North Metro. A slower ramp affects three facilities and is concentrated among children, adolescents, and younger adults. This pattern is intended to test EWMA and cluster-based detection.

Facility reporting artifact at Facility_12. Several days of reduced reporting are followed by a batch upload. This creates a strong artificial spike that should be interpreted as a data quality issue rather than an epidemiologic event.

Older-adult respiratory severity signal in East River. ED visits rise modestly while hospitalizations among older adults rise more sharply. This tests severity-oriented interpretation.

5.4 Data Quality Flags

Synthetic data quality flags include:

- `missing_chief_complaint,`
- `missing_diagnosis,`
- `duplicate_message,`
- `delayed_report,`
- `facility_batch_upload,`
- `inconsistent_label.`

These flags are persisted on synthetic messages, parsed encounters, and surveillance metrics. They are also surfaced in the ML signal detection page so analysts can separate operational artifacts from possible disease activity.

6. HL7/EHR Processing

The system generates synthetic ADT_A01-style messages containing:

- `MSH,`
- `PID,`
- `PV1,`
- `OBX,`
- `DG1.`

The parser extracts structured fields such as region, facility, chief complaint, diagnosis code, test result, encounter date, and report date. The HL7 page demonstrates how raw message content can be connected to downstream surveillance metrics. This is important because signal interpretation often depends on message quality, missing fields, delays, and duplicate events.

The parser is intentionally lightweight. It is designed to illustrate data flow and validation logic rather than replace enterprise HL7 interface engines.

7. Feature Engineering

Feature engineering aggregates surveillance metrics by date, region, facility, syndrome, and age group. The engineered features include:

- ED visit count,
- mean test positivity,
- hospitalization count,
- death count,
- reporting delay,
- data quality score,
- data quality flag count,
- day of week,
- rolling 7-day and 14-day baselines,
- rolling standard deviations,
- z-scores,
- percent change from rolling baseline,
- hospitalization rolling baseline,
- death rolling baseline,
- test positivity rolling baseline,
- EWMA value,
- EWMA delta,
- affected facility count,
- affected age-group count.

Categorical fields are one-hot encoded for the Isolation Forest feature matrix. Numeric features are standardized before unsupervised modeling.

8. Anomaly Detection Methods

The anomaly detection layer combines several complementary methods. Each method contributes evidence, and the final result is interpreted through model agreement and public health context.

8.1 Rolling Baseline Z-Score

Rolling z-scores compare current ED visits to shifted 7-day and 14-day historical baselines. Shifting prevents the current observation from being included in the baseline. This method is easy to explain and useful for sudden spikes.

Strengths:

- transparent,
- fast,
- interpretable,
- useful for abrupt changes.

Limitations:

- sensitive to small denominators,
- less effective for slow ramps,
- can overreact to reporting artifacts.

8.2 EWMA Acceleration

EWMA smooths recent trends while preserving responsiveness. EWMA delta is used to detect acceleration. This helps identify gradual increases that may not generate extreme one-day z-scores.

Strengths:

- useful for slow-moving outbreaks,
- less noisy than raw daily counts,
- easy to compare over time.

Limitations:

- can lag sudden spikes,
- still depends on stable historical reporting.

8.3 Isolation Forest

Isolation Forest is used as a multivariate outlier detector. It receives numeric indicators, rolling features, quality context, and encoded categorical fields.

Strengths:

- detects unusual combinations of features,
- captures multivariate anomalies,
- does not require labeled outbreak data.

Limitations:

- anomaly scores are not calibrated probabilities,
- interpretation requires supporting evidence,
- contamination settings influence sensitivity.

8.4 DBSCAN

DBSCAN identifies outliers and cluster structure in surveillance feature space. A DBSCAN outlier is treated as supporting evidence, especially when it aligns with rolling baseline or EWMA signals.

Strengths:

- can identify isolated abnormal points,
- no need to predefine number of clusters,
- useful for cluster/outlier context.

Limitations:

- sensitive to feature scaling and parameter choices,
- cluster labels are not epidemiologic diagnoses.

9. Signal Scoring and Classification

The anomaly score combines:

- maximum positive z-score,
- Isolation Forest flag,
- DBSCAN outlier flag,
- EWMA acceleration flag,
- test positivity lead indicator,
- hospitalization lag indicator,
- affected facilities,
- affected age groups,
- deaths,
- and quality flag penalties.

Severity score further incorporates model agreement and severity indicator changes. Severity labels are assigned as Low, Moderate, High, or Critical based on score thresholds.

Likely signal type is assigned using interpretable rules:

- `reporting artifact` when data quality flags, batch upload, or delay context dominates,
- `likely true outbreak` when positivity and hospitalization evidence are coherent,
- `severity signal` when hospitalizations rise disproportionately,
- `undetermined signal` when evidence is incomplete or mixed.

This rule layer is not a substitute for epidemiologist judgment. It is a triage aid that makes the assumptions visible.

10. Forecasting

The forecasting module produces short-horizon ED visit projections by syndrome. The method uses a recent moving average and trend estimate, then widens uncertainty after day 7.

The forecast output includes:

- `date`,

- horizon group,
- predicted ED visits,
- lower bound,
- upper bound,
- interpretation,
- and method description.

The frontend supports 7-day and 14-day horizons and displays the forecast interval visually. The method is intentionally simple because the prototype focuses on workflow integration and interpretability. Future versions could compare this baseline with ARIMA, Prophet-style models, LSTM, temporal convolution, or transformer-based approaches.

11. Retrieval-Augmented Public Health Guidance

The RAG subsystem indexes a local markdown knowledge base with ChromaDB. Documents cover syndromic surveillance, test positivity, hospitalization lag, death lag, chief complaint analysis, reporting delays, facility spikes, data quality problems, explainable AI, and analyst workflow.

The RAG endpoint returns:

- a concise public health interpretation,
- supporting snippets,
- source documents,
- investigation checklist,
- related indicators to review.

The RAG panel is intentionally not framed as a chatbot. It is an analyst support tool that retrieves relevant context and recommends review steps.

Optional OpenAI mode can use OpenAI embeddings and response generation. A fine-tuned model can be used to control response structure and caution language, while the factual grounding still comes from retrieved documents.

12. Frontend Workflow Design

The user interface is designed for repeated analytical review. Recent enhancements include:

- descriptive dashboard chart titles and time frames,
- region names instead of generic region IDs,
- multi-line syndrome trend charts,
- forecast horizon selection and uncertainty summaries,
- ML detection filters,
- organized anomaly evidence columns,
- model chips,
- quality flag chips,

- and summary cards for current filtered views.

The design favors dense but readable operational interfaces over marketing-style presentation. Cards are used for repeated metrics and framed analytical tools. Charts include explicit axes, legends, and tooltips.

13. API Surface

The API endpoints support the full research workflow:

Endpoint	Purpose
POST /api/generate-synthetic-data	Generate or reuse synthetic data
GET /api/records	Return parsed encounters
GET /api/dashboard/summary	Return latest dashboard metrics
GET /api/dashboard/trends	Return trend and comparison data
GET /api/hl7/messages	Return raw synthetic HL7 messages
POST /api/hl7/parse	Parse submitted HL7-style content
POST /api/ml/run-detection	Run anomaly detection
GET /api/ml/anomalies	Return ranked anomaly rows
GET /api/ml/forecast	Return syndrome forecast
POST /api/rag/query	Query knowledge base
POST /api/rag/reindex	Rebuild vector index
GET /api/rag/status	Return RAG configuration
POST /api/insights/generate	Generate structured anomaly insight
POST /api/reports/generate	Generate investigation brief

14. Performance Characteristics

The prototype is designed for local interactive research. The default dataset is large enough to demonstrate realistic stratification but small enough to run on a laptop.

Expected performance characteristics:

- Data generation is deterministic and reproducible.
- Aggregations use pandas groupby operations.
- Rolling features are computed over grouped time series.
- Unsupervised models operate on engineered metric rows.

- Dashboard endpoints return aggregated data rather than raw tables.
- RAG indexing is local and suitable for a compact knowledge base.

The current implementation has not been benchmarked under production load. It should be treated as a research prototype, not a scale-tested public health platform.

15. Evaluation Strategy

A future validation plan should include:

- 1 Detection timeliness against known injected scenario onset dates.
- 2 False positive analysis during known reporting artifact periods.
- 3 Precision and recall using scenario labels as synthetic ground truth.
- 4 Stratified evaluation by region, facility size, syndrome, and age group.
- 5 Sensitivity analysis for z-score thresholds, Isolation Forest contamination, DBSCAN parameters, and EWMA span.
- 6 Runtime benchmarking for data generation, detection, forecasting, RAG indexing, and dashboard endpoints.
- 7 Human factors evaluation with public health analysts or domain reviewers.

Because the data is synthetic, these metrics would evaluate system behavior under controlled assumptions. They would not establish operational validity.

16. Security, Privacy, and Ethics

The platform avoids real patient data. This reduces privacy risk and makes the repository suitable for public demonstration. However, any real-world adaptation would require:

- privacy review,
- security controls,
- role-based access control,
- audit logging,
- data governance,
- interface validation,
- clinical/public health oversight,
- model monitoring,
- and formal evaluation.

The system should not automate outbreak confirmation. Its outputs should be interpreted as decision support requiring expert review.

17. Limitations

Key limitations include:

- No real-world validation.
- No calibrated outbreak probabilities.
- Synthetic message formats are simplified.
- Forecasting is intentionally basic.
- RAG guidance depends on local document quality.
- Model thresholds are heuristic.
- No production authentication or authorization layer.
- No streaming ingestion.
- No formal epidemiologic performance evaluation.

These limitations are acceptable for a research prototype but must be addressed before operational use.

18. Future Work

Recommended future work includes:

- FHIR resource support.
- Real-time streaming ingestion.
- Advanced chief complaint NLP classification.
- Probabilistic outbreak scoring.
- Bayesian or hierarchical baselines for small facilities.
- LSTM or transformer-based forecasting comparison.
- Graph-based region/facility propagation analysis.
- Automated benchmark reports.
- Role-based access control.
- Cloud deployment reference architecture.
- Human-in-the-loop review and feedback capture.

19. Conclusion

The AI-Assisted Public Health Surveillance Platform demonstrates a practical architecture for combining synthetic surveillance data, interpretable anomaly detection, short-term forecasting, retrieval-augmented guidance, and explainable reporting. Its central contribution is not a novel algorithm, but an integrated workflow that makes model evidence, data quality context, and public health interpretation visible to analysts.

The prototype shows how AI/ML systems for public health can be designed around transparency, reproducibility, and human review. It provides a foundation for future validation, improved forecasting, richer NLP, and operational hardening while maintaining a clear boundary: synthetic research decision support, not automated outbreak confirmation.